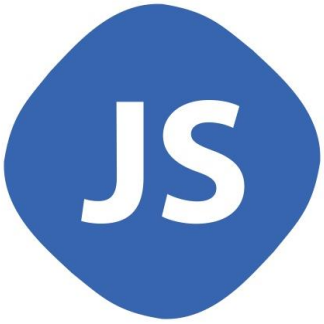


Замыкания

- Введение
- Потеря контекста
- Поиск контекста
- Контекст и callback
- Что такое замыкание
- Практика применения замыканий
- Контроль побочных эффектов
- Приватные переменные

Введение

Предыдущие уроки были важными, но настоящую их ценность можно будет почувствовать только сейчас.

> ***Примечание***. Для качественного усвоения материала уроки:
>
> - **Область видимости и лексическое окружение**
> - **Лексическое окружение и контекст**
>
> **обязательны** к изучению.

Сегодня рассмотрим замыкания. В глобальной сети материала, посвященного замыканиям достаточно, но тем не менее, вопросов возникает много. И я бы не сказал, что замыкания – это очень сложно, скорее трудно понять зачем это нужно и где/как это использовать. Использовать – это значит перенести приобретенный опыт использования замыканий на свои проекты.

> ***Примечание***. Некоторые разделы урока переплетаются настолько, что трудно определить, что из чего вытекает. Поэтому мой совет – рассматривайте каждый раздел, как самостоятельный материал на тему

замыканий.

Конечная цель урока разобраться со следующими вопросами:

- **Потеря** контекста в методах объекта.
- **Восстановление** контекста:
  - функция-**обертка**
  - метод **bind**
- Что такое **замыкание**. Суть замыканий.
- **Практика** применения замыканий.

Потеря контекста

В этом разделе делаем упор на разбор ситуаций, при которых может **теряться** контекст **метода** (функции) **объекта**. Как только метод передаётся отдельно от объекта – **this** используемый в функции-методе объекта теряется.

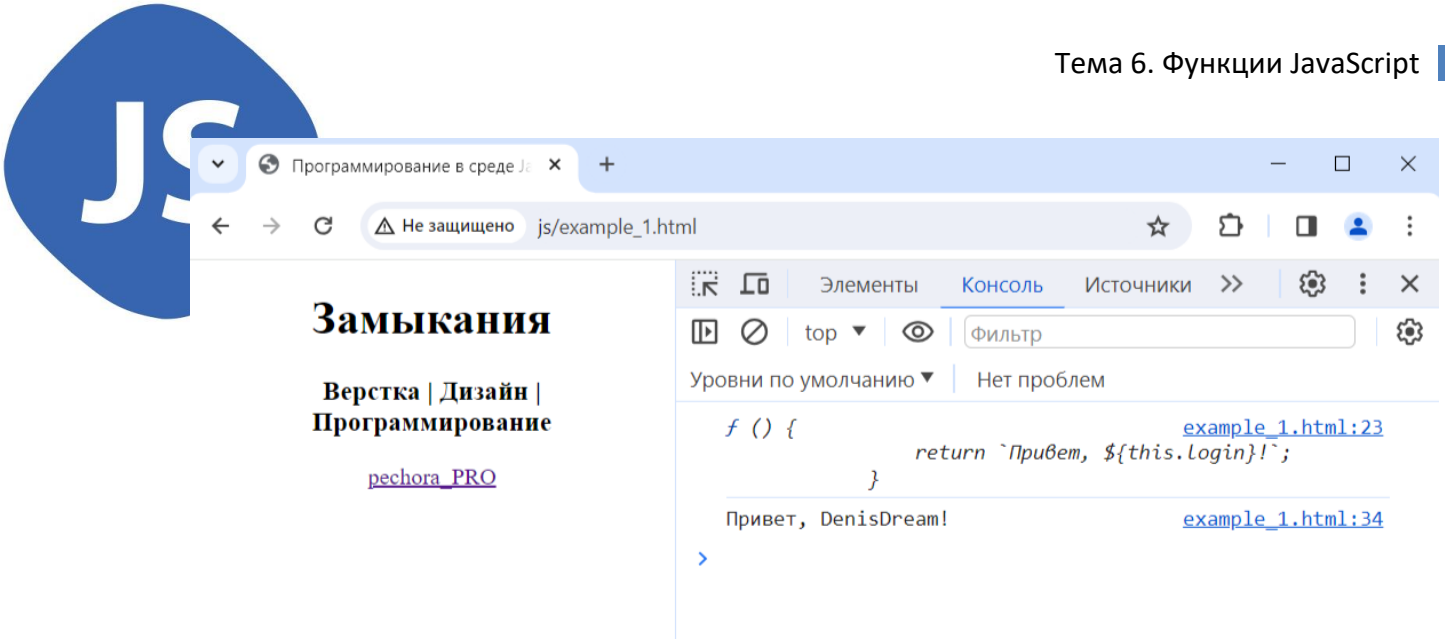
Прежде, чем приступить к изучению причин потери контекста, разберем вводный пример. В демонстрационном примере **example_1** особое внимание обратите на следующие строки кода:

```
// вывод определения метода объекта в консоль
console.log(user.welcome);
/*
    консоль:
    ƒ () {
        return `Привет, ${this.login}!`;
    }
*/
```

Именно на понимании семантики данной **синтаксической** конструкции будет построен весь урок. К этой конструкции в той или иной степени я буду обращаться на протяжении всего урока.

example_1.

```
<script>
    // определение объекта
    let user = {
        login: "DenisDream",
        // метод оббъекта
        welcome: function() {
            return `Привет, ${this.login}!`;
        }
    };
    // вывод определения метода объекта в консоль
    console.log(user.welcome);
    /*
        консоль:
        ƒ () {
            return `Привет, ${this.login}!`;
        }
    */
    // user.welcome <> user.welcome()
    // запуск метода в контексте объекта
    console.log(user.welcome());
    /*
        консоль:
        Привет, DenisDream!
    */
</script>
```

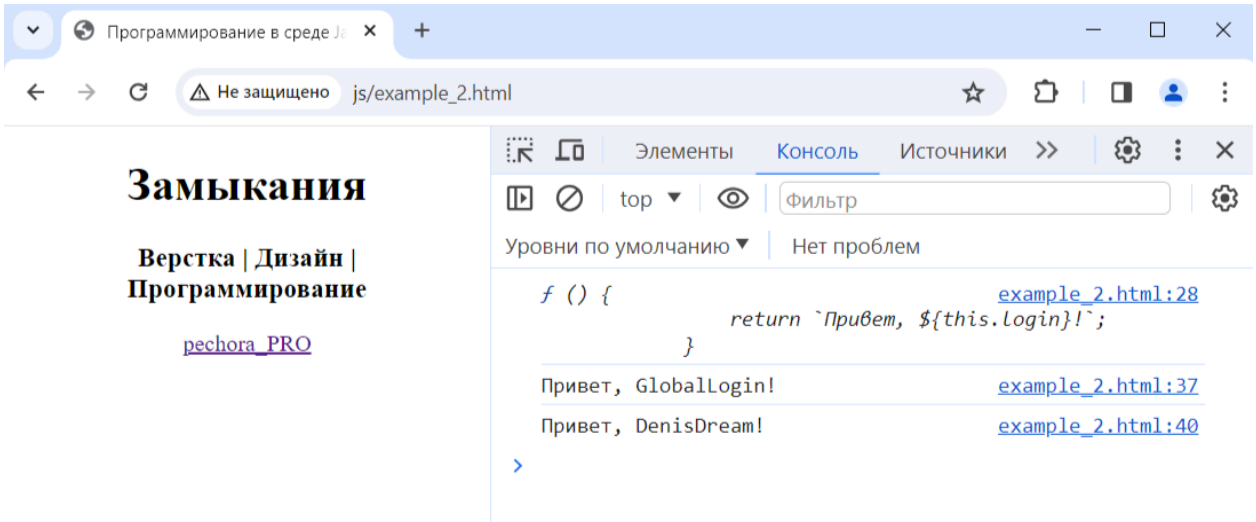


Потеря контекста в глобальном объекте window

example_2.

```
<script>
    // глобальная переменная
    var login = "GlobalLogin";
    // объект
    let user = {
        login: "DenisDream",
        // метод объекта
        welcome: function () {
            return `Привет, ${this.login}!`;
        }
    };
    let msg = user.welcome;
    // просмотр содержимого переменной msg
    console.log (msg);
    /*
```

```
        консоль:
        f () {
            return `Привет, ${this.login}!`;
        }
    */
    // потеря контекста в глобальном объекте window
    console.log (msg()); // Привет, GlobalLogin!
    // запуск метода в контексте объекта
    console.log(user.welcome()); // Привет, DenisDream!
</script>
```



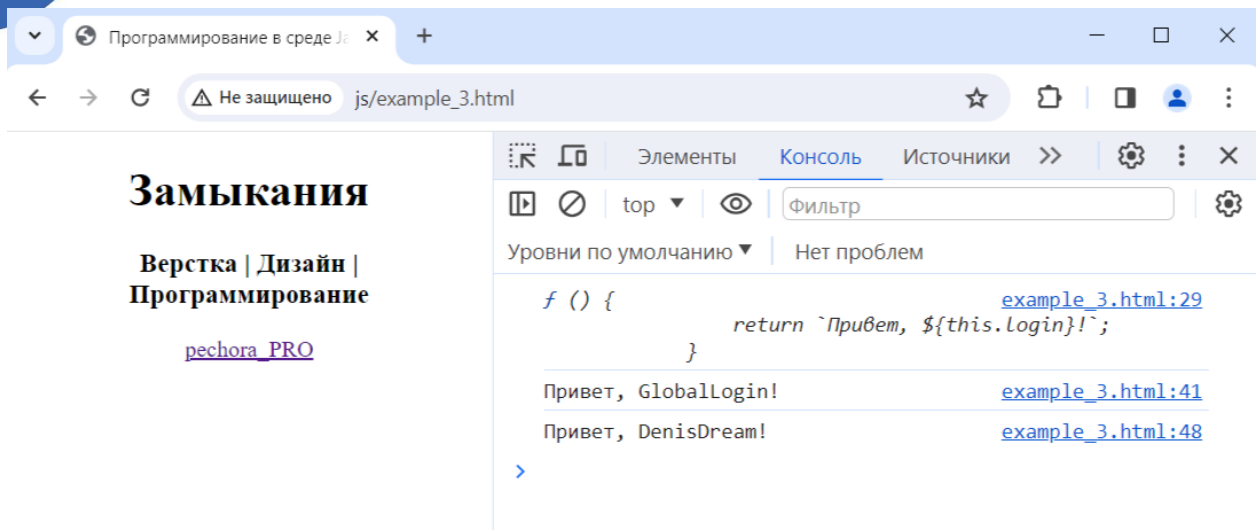
Потеря контекста в функции с аргументом callback

example_3.

```
<script>
    // глобальная переменная
    var login = "GlobalLogin";
    // объект
```

```
let user = {
    login: "DenisDream",
    // метод объекта
    welcome: function() {
        return `Привет, ${this.login}!`;
    }
};
// описание функции
function fnTestCallback (callback) {
    // login = "None";
    console.log(callback);
    /*
    консоль:
    ƒ () {
        return `Привет, ${this.login}!`;
    }
    */
    // вызов функции callback
    // callback = welcome()
    // callback() = welcome()()
    console.log(callback()); // Привет, GlobalLogin!
}
// потеря контекста в функции с аргументом callback
fnTestCallback(user.welcome);
// запуск метода в контексте объекта
console.log(user.welcome()); // Привет, DenisDream!
```

```
</script>
```



Потеря контекста в асинхронном вызове

example_4.

```
<script>

    // глобальная переменная

    var login = "GlobalLogin";

    // объект

    let user = {

        login: "DenisDream",

        // метод

        welcome: function() {

            alert (`Привет, ${this.login}!`);

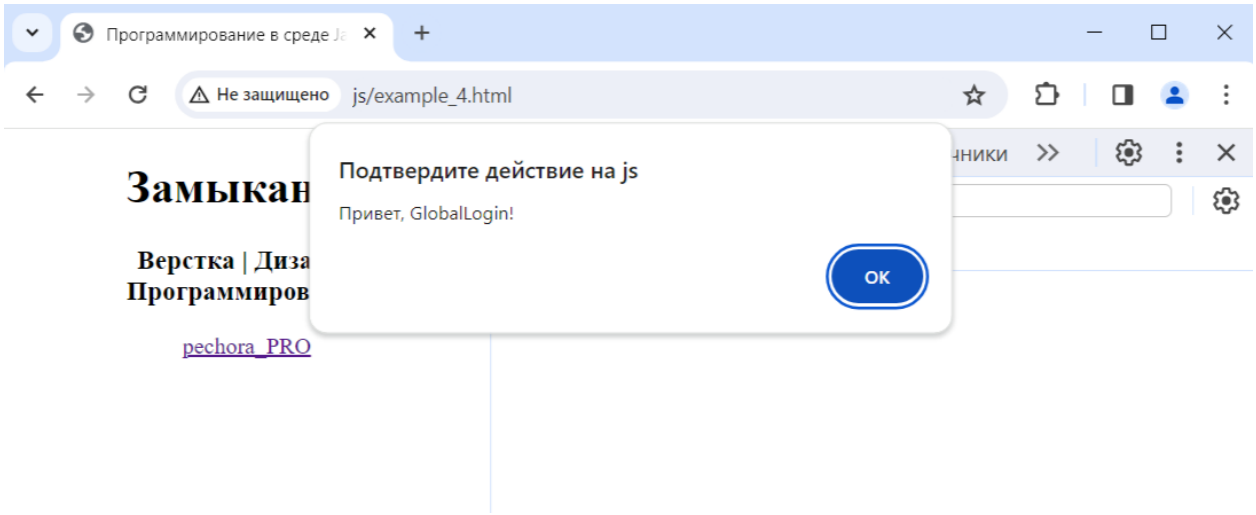
        }

    };

    // потеря контекста в асинхронном вызове

    setTimeout(user.welcome, 3000); // Привет, GlobalLogin!
```

```
    // запуск метода в контексте объекта
    user.welcome(); // Привет, DenisDream!
</script>
```

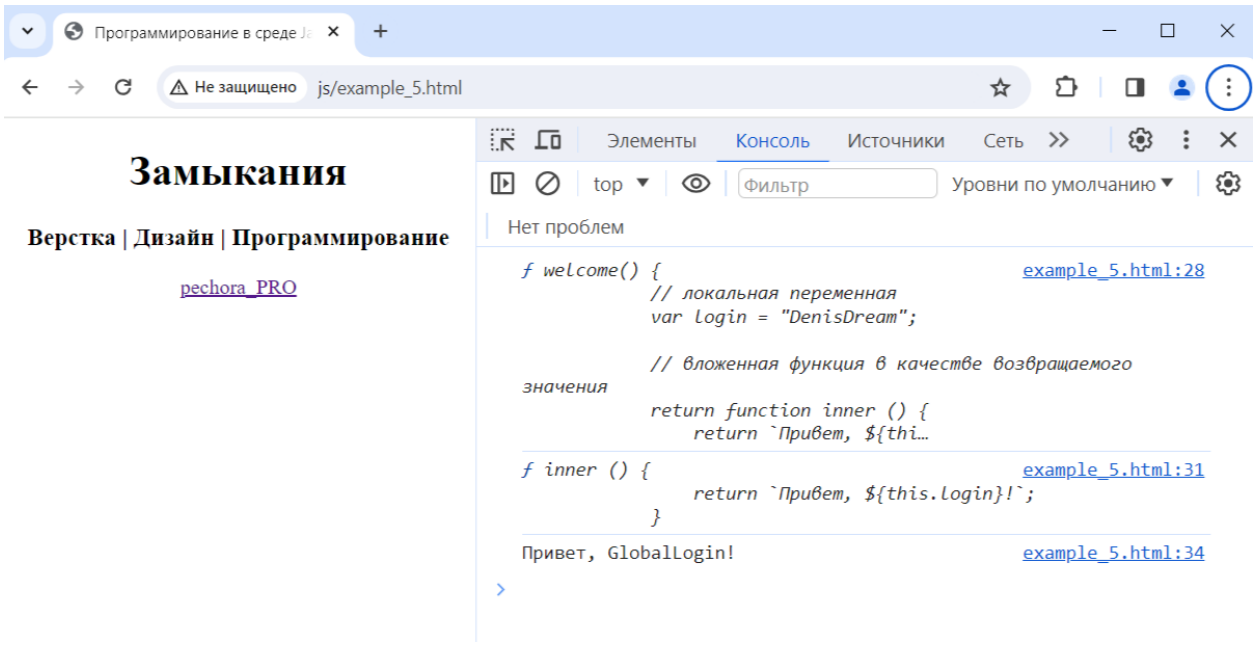


Обратите внимание, следующую ситуацию нельзя считать потерей контекста – это **штатное поведение** функций.

example_5.

```
<script>
    // глобальная переменная
    var login = "GlobalLogin";
    // определение функции
    function welcome() {
        // локальная переменная
        var login = "DenisDream";
        // вложенная функция в качестве возвращаемого значения
        return function inner () {
            return `Привет, ${this.login}!`;
        }
```

```
    };
    // определение внешней функции welcome
    console.log(welcome);
    // определение вложенной функции inner
    console.log(welcome());
    // запуск функции welcome
    console.log(welcome()());
</script>
```



Это потеря контекста. Теперь рассмотрим способы его поиска :).

Важно. Решить проблему потери контекста можно с помощью **функции обертки** или с помощью методов **call**, **bind**, **apply** (в зависимости от цели).



Поиск контекста

В этом разделе делаем упор на **восстановление** утраченного контекста.

example_6.

```
<script>
    // глобальная переменная
    login = "JimMorrison";
    // объект
    var user = {
        login: "DenisDream",
        pwd: "12345",
        role: "admin",
        // метод объекта
        welcome: function() {
            alert(`Привет, ${this.login}!`);
        }
    };
    // сработает через 2 сек.
    // потеря контекста
    setTimeout (user.welcome, 2000);
    // сработает сразу
    // это вообще не то ...
    setTimeout (user.welcome(), 2000);
</script>
```





Восстановление контекста функцией-обёрткой

example_7.

```
<script>
    // глобальная переменная
    login = "JimMorrison";
    // объект
    var user = {
        login: "DenisDream",
        pwd: "12345",
        role: "admin",
        // метод объекта
        welcome: function() {
            alert(`Привет, ${this.login}!`);
        }
    };
    // обернем вызов user.welcome() в функцию
    setTimeout( () => {user.welcome()}, 2000);
</script>
```

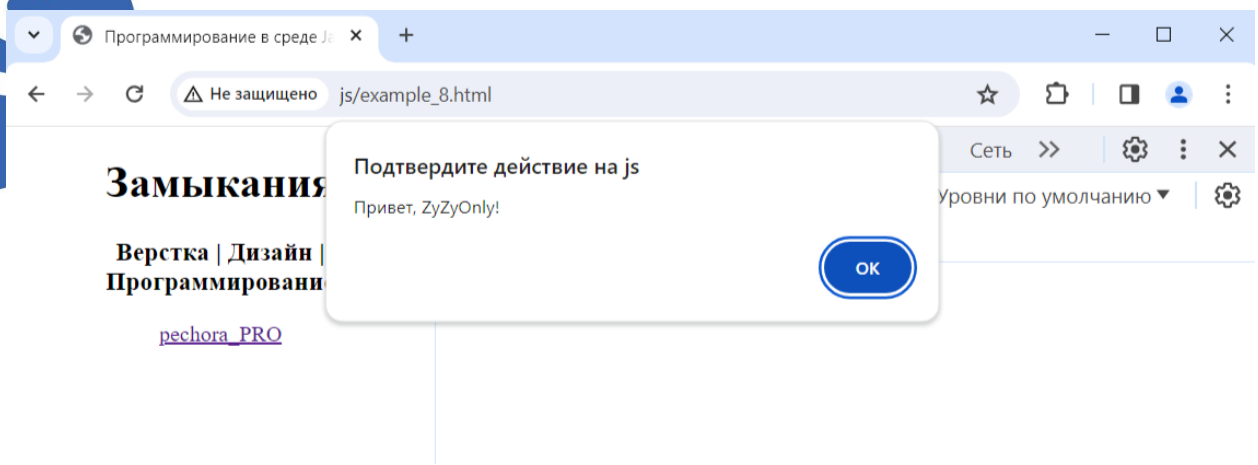
Вроде бы проблема потери контекста решена. Но попробуйте выполнить код демонстрационного примера **example_8**. Суть примера в следующем – за время ожидания выполнения отложенного кода функцией **setTimeout**(), объект **user** изменился.

example_8.

```
<script>
    // глобальная переменная
    login = "JimMorrison";
```



```
    // объект user
    var user = {
        login: "DenisDream",
        pwd: "12345",
        role: "admin",
        welcome: function() {
            alert(`Привет, ${this.login}!`);
        }
    };
    // функция callback для setTimeout() сработает через 2 сек.
    // но к этому времени объект user будет переопределен
    setTimeout( () => {user.welcome()}, 2000);
    // переопределили объект user
    var user = {
        login: "ZyZyOnly",
        pwd: "qwerty",
        role: "moderator",
        welcome: function() {
            alert(`Привет, ${this.login}!`);
        }
    };
</script>
```



Восстановление контекста методом bind

Проблему демонстрационного примера **example_8** решим специальными методами привязки контекста.

Обратите внимание на **консольный** вывод:

```
// сохраним контекст с помощью метода bind
let welcome = user.welcome.bind(user);
console.log(welcome);
/*
    консоль:
    ƒ () {
        alert(`Привет, ${this.login}!`);
    }
*/
```

В демо примере **example_10** не только **сохраним** контекст, но и запишем его в **массив**. Ну а пока, сохраним контекст в переменную **welcome**.

example_9.

```
<script>
```

```
// глобальная переменная
login = "JimMorrison";
// объект user
var user = {
    login: "DenisDream",
    pwd: "12345",
    role: "admin",
    welcome: function() {
        alert(`Привет, ${this.login}!`);
    }
};
// сохраним контекст с помощью метода bind
let welcome = user.welcome.bind(user);
console.log(welcome);
/*
    консоль:
    ƒ () {
        alert(`Привет, ${this.login}!`);
    }
*/
// переопределили объект user
var user = {
    login: "ZyZyOnly",
    pwd: "qwerty",
    role: "moderator",
    welcome: function() {
```

```
            alert(`Привет, ${this.login}!`);
        }
    };
    // вызов welcome() с привязкой к объекту user
    setTimeout( () => {welcome()}, 2000);
</script>
```

Продвинутая версия демо **example_9** представлена в примере **example_10**. Обратите внимание на **сохранение** в массив и последующий **вызов** функций, созданных методом **bind**.

example_10.

```
<script>
    // массив для хранения функций, созданных методом bind
    let welcome = new Array();
    // объект user
    var user = {
        login: "DenisDream",
        pwd: "12345",
        role: "admin",
        welcome: function() {
            alert(`Привет, ${this.login}!`);
        }
    };
    // сохраним вызов метода с привязкой к объекту
    welcome.push(user.welcome.bind(user));
    // переопределили объект user
    var user = {
```

```
        login: "ZyZyOnly",
        pwd: "qwerty",
        role: "moderator",
        welcome: function() {
            alert(`Привет, ${this.login}!`);
        }
    };
    // сохраним вызов метода с привязкой к объекту
    welcome.push(user.welcome.bind(user));
    // через 3 сек. выполним вызов сохраненных функций
    setTimeout( () => {
        for (var i=0; i<welcome.length; i++) {
            welcome[i]();
        }
    }, 3000);
</script>
```

Контекст и callback

В этом разделе делаем упор на **восстановление** контекста в **callback** функциях.

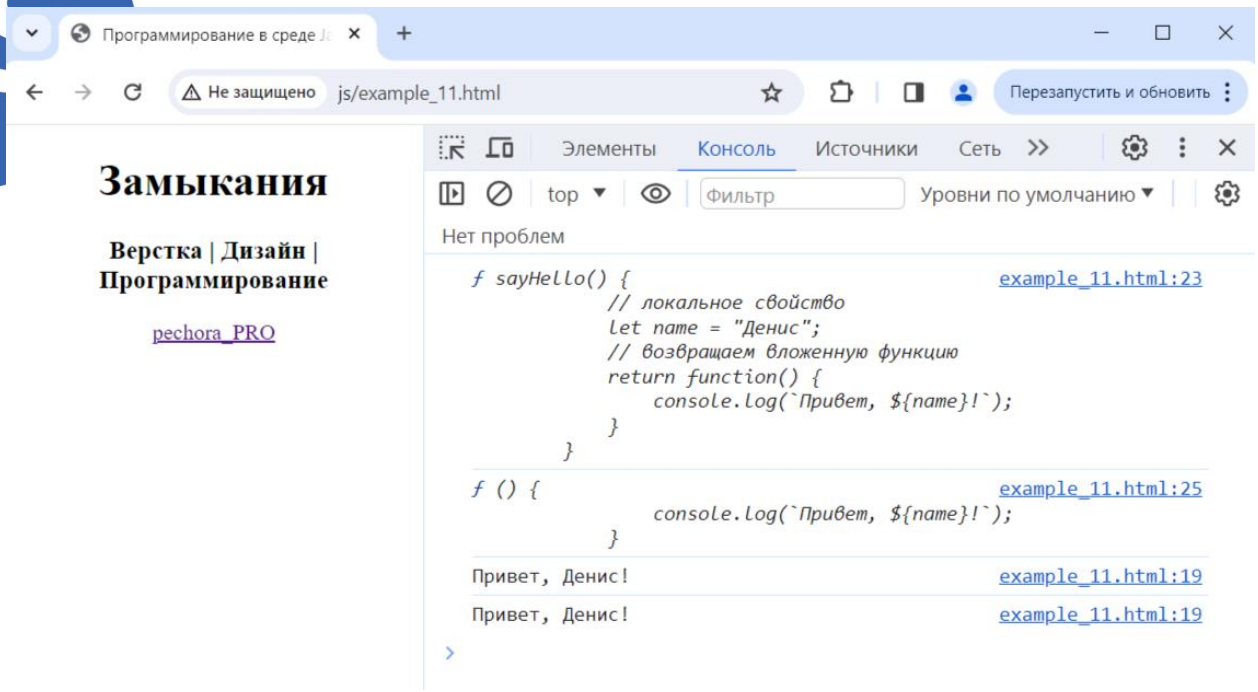
В демонстрационном примере **example_11** вспомним, что может возвращать одна и таже функция в **разных режимах вызова**. Очень важная информация для усвоения заявленных вопросов текущего урока.

example_11.

```
<script>
```

```
    // внешняя функция
    function sayHello() {
        // локальное свойство
        let name = "Денис";
        // возвращаем вложенную функцию
        return function() {
            console.log(`Привет, ${name}!`);
        }
    }
    console.log(sayHello);
    console.log(sayHello());
    sayHello()();
    outUser(sayHello);
    // callback - метод объекта
    function outUser(callback) {
        // вызов вложенной функции callback
        callback()();
    }
</script>
```

Консольный вывод примера представлен на следующем скриншоте.



А теперь рассмотрим проблему потери контекста именно в **callback**. Ну и способы его восстановления.

Потеря контекста в callback

example_12.

```
<script>
    // глобальная переменная
    name = "GlobalName";
    // объект
    let user = {
        // свойство объекта
        name: "Денис",
        // метод объекта
        userInfo: function() {
            console.log(`Привет, ${this.name}!`);
        }
```

```
    }
    // вызов метода объекта
    user.userInfo(); // Привет, Денис!
    // вызов функции с аргументом - callback
    outUser(user.userInfo); // Привет, GlobalName!
    // аргумент функции - callback (метод объекта)
    function outUser(callback) {
        // вызов функции вне контекста объекта
        callback();
    }
</script>
```

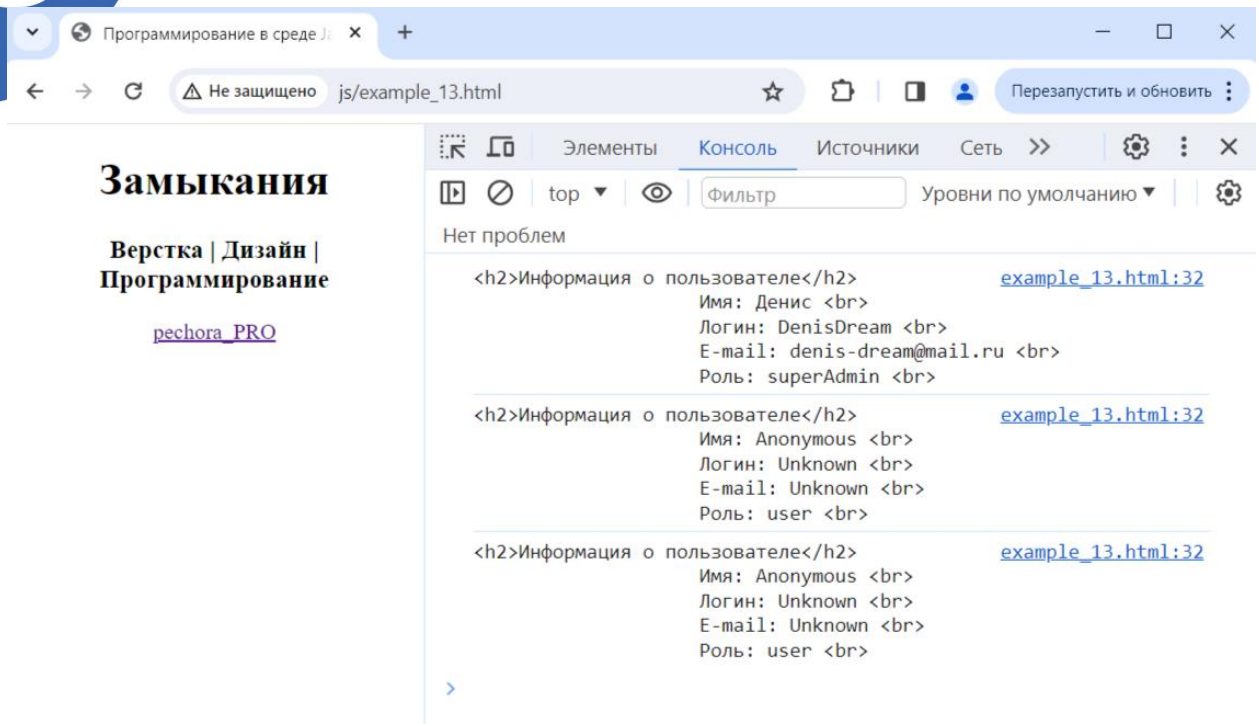
В демонстрационном примере **example_13** потеря контекста в функции с **callback** и в асинхронном **setTimeout**().

example_13.

```
<script>
    // глобальные переменные
    name = "Anonymous";
    login = "Unknown";
    email = "Unknown";
    role = "user";
    // объект
    let user = {
        name: "Денис",
        login: "DenisDream",
        email: "denis-dream@mail.ru",
```

```
        role: "superAdmin",
        userInfo: function() {
            let str = `
                <h2>Информация о пользователе</h2>
                Имя: ${this.name} <br>
                Логин: ${this.login} <br>
                E-mail: ${this.email} <br>
                Роль: ${this.role} <br>
            `;
            console.log(str);
        }
    }
    // аргумент функции - callback (метод объекта)
    function outUser(callback) {
        // вызов callback
        callback();
    }
    // вызов метода объекта
    user.userInfo();
    // передача кода метода (callback)
    // потеря контекста this
    outUser(user.userInfo);
    // передача кода метода (callback)
    // потеря контекста this
    setTimeout(user.userInfo, 1000);
</script>
```

Результат потери контекста продемонстрирован на скриншоте.



Решаем проблемы уже **известными нам способами**.

Восстановление контекста методом call

example_14.

```
<script>

    // глобальные переменные

    name = "Anonymous";

    login = "Unknown";

    email = "Unknown";

    role = "user";

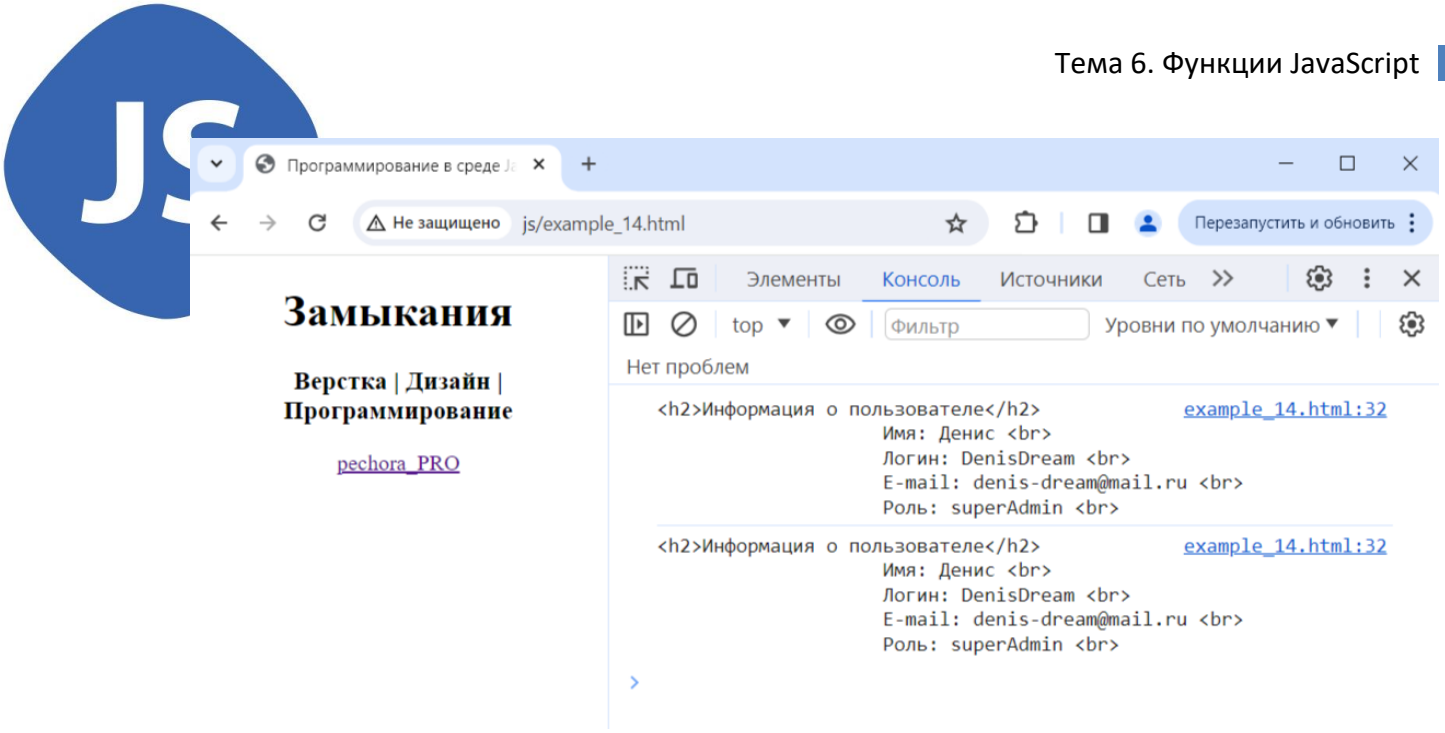
    // объект

    let user = {

        name: "Денис",

        login: "DenisDream",
```

```
        email: "denis-dream@mail.ru",
        role: "superAdmin",
        userInfo: function() {
            let str = `
                <h2>Информация о пользователе</h2>
                Имя: ${this.name} <br>
                Логин: ${this.login} <br>
                E-mail: ${this.email} <br>
                Роль: ${this.role} <br>
            `;
            console.log(str);
        }
    }
    // аргумент функции - callback (метод объекта)
    function outUser(callback) {
        // callback();
        callback.call(user);
    }
    // вызов функции с callback
    outUser(user.userInfo);
    // вызов метода объекта
    user.userInfo();
</script>
```



Восстановление контекста функцией-обёрткой

example_15.

```
<script>

    // глобальные переменные

    name = "Anonymous";

    login = "Unknown";

    email = "Unknown";

    role = "user";

    // объект

    let user = {

        name: "Денис",

        login: "DenisDream",

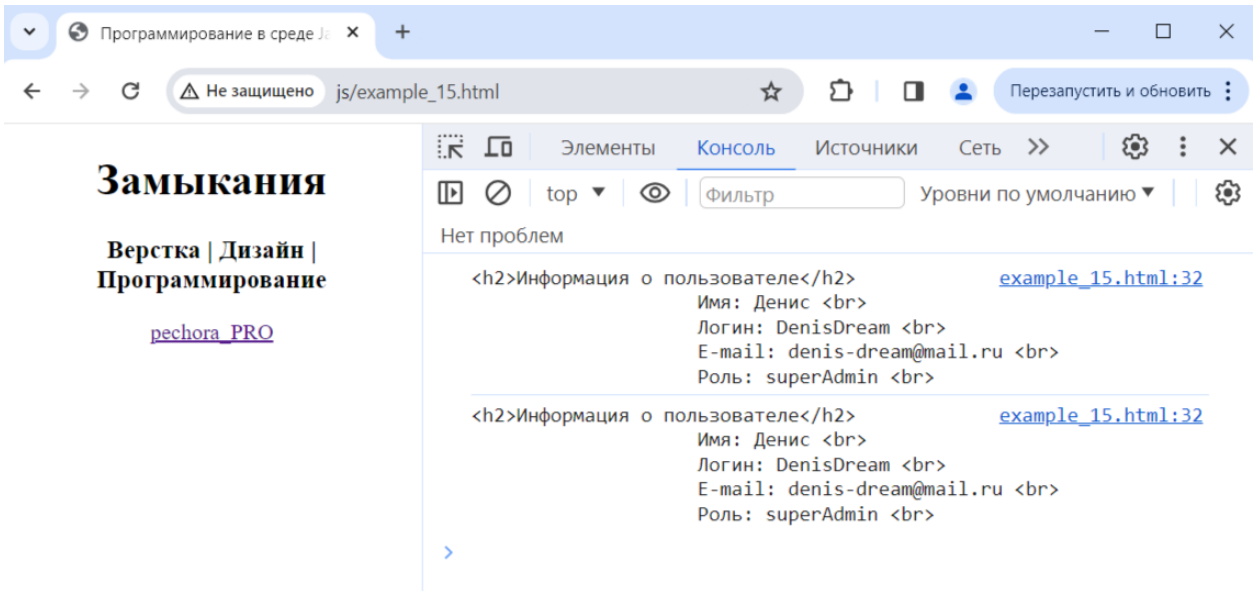
        email: "denis-dream@mail.ru",

        role: "superAdmin",

        userInfo: function() {

            let str = `
```

```
                <h2>Информация о пользователе</h2>
                Имя: ${this.name} <br>
                Логин: ${this.login} <br>
                E-mail: ${this.email} <br>
                Роль: ${this.role} <br>
            `;
            console.log(str);
        }
    }
    // передача кода метода (callback)
    setTimeout( () => {user.userInfo()}, 1000);
    // вызов метода объекта
    user.userInfo();
</script>
```





Что такое замыкание

Мы рассмотрели разные ситуации потери контекста и его восстановления. При этом использовали замыкания. **Так что же такое замыкание?**

Для того, чтобы разобраться с замыканиями необходимо хорошо представлять, что такое глобальная (вложенная) **лексическая область видимости**. Рассмотрим суть возникновения **замыкания** в нескольких взаимосвязанных примерах.

example_16.

```
<script>
    // внешняя функция
    function outer() {
        // внутренняя функция
        function inner() {
            console.log('Привет!');
        }
        // инструкции return нет
    }
    // вызов внешней функции outer
    console.log(outer());
    /*
        консоль:
        undefined (неявный)
    */
</script>
```



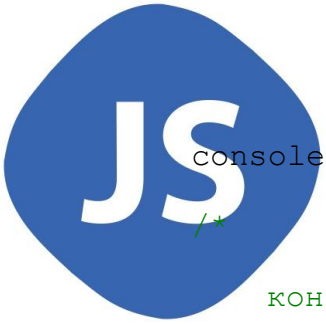
Хорошим примером использования **замыкания** является **возврат ссылки на функцию**. Мы можем возвращать наружу разные ссылки, которые делают возможным доступ к тому, что было определено внутри.

example_17.

```
<script>
    // внешняя функция
    function outer() {
        // возвращаем определение функции inner
        return function inner() {
            console.log('Привет!');
        }
    }
    // вывод в консоль определения вложенной функции
    console.log(outer());
    /*
        консоль:
        ƒ inner () {
            console.log('Привет!');
        }
    */
    // вызов вложенной функции
    // такие вызовы и есть замыкания
    outer()();
</script>
```

Чтобы вывести на экран текст, недостаточно просто вызвать функцию **outer**().

```
// вывод в консоль определения вложенной функции
```



```
console.log(outer());
/*
    консоль:
    ƒ inner () {
        console.log('Привет!');
    }
*/
```

Функция возвращает функцию, поэтому возвращаемое значение надо еще раз вызвать, например так:

```
// вызов вложенной функции
// такие вызовы и есть замыкания
outer()();
```

Такие вызовы и есть замыкания. Замыкание можно вызвать напрямую, а можно вызвать через **присваивание переменной**, например так:

```
// сохранение состояния (лексического окружения)
// создание замыкания
let hi = outer();
// вызов замыкания
hi();
```

example_18.

```
<script>
    // внешняя функция
    function outer() {
        // возвращаем определение безымянной функции
        return function () {
            console.log('Привет!');
```

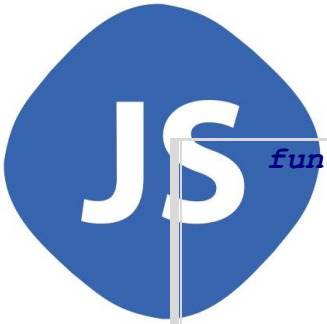
```
    }
}
// сохранение состояния (лексического окружения)
// создание замыкания
let hi = outer();
// вывод замыкания
console.log(hi);
/*
    консоль:
    ƒ () {
        console.log('Привет!');
    }
*/
// вызов замыкания
hi();
</script>
```

Важно. Чтобы быть замыканием функции не обязательно что-то возвращать. Любой доступ к переменным извне текущей области видимости **создаёт замыкание**.

Но гораздо интереснее следующий пример, когда происходит **сохранение переменных лексического окружения**.

example_19.

```
<script>
    // внешняя функция
```



```
function outer(name) {
    let email = "denis-dream@mail.ru";
    let role = "admin";
    /*
        локальные переменные функции outer:
        - name
        - email
        - role
    */
    // возвращаем определение безымянной функции
    return function() {
        // вложенная функция имеет доступ к лексическому
        окружению родителя
        document.write(`
            <h3>Привет, ${name}!</h3>
            Вы зашли с ролью <b>${role}</b><br>
            Ваш e-mail: <b>${email}</b>
        `);
    }
}
/*
сохранение состояния лексического окружения
    - name
    - email
    - role
*/
```

```
    // создание замыкания
    let hi = outer('Денис');
    // переменные глобального контекста
    var name = "Anonymous";
    var email = "Unknown";
    var role = "user";
    // вызов замыкания
    hi();
</script>
```

Всякий раз выполняя код, обращайте внимание на консольный вывод (вывод в браузер), или организуйте его самостоятельно. Анализируйте, разбирайте предлагаемые примеры.

example_20.

```
<script>
    // внешняя функция
    function outer(num) {
        // локальный объект
        user = [
        {
            login: "web-master",
            pwd: "qwerty",
            email: "web-master@my-site.org",
        },
        {
            login: "scill-free",
            pwd: "12345",
```

```
        email: "scill-free@my-site.org",
    }
    ]
    // возвращаем определение безымянной функции
    return function() {
        with (user[num]) {
            document.write (`
                <h3>Уважаемый пользователь, вы запросили
                персональные данные:</h3>
                Логин: ${login} <br>
                Пароль: ${pwd} <br>
                E-mail: ${email} <br>
            `);
        }
    }
}
// переменные глобального контекста
var name = "Anonymous";
var email = "Unknown";
var role = "user";
// создание замыкания
let person_1 = outer(0);
let person_2 = outer(1);
// вызов замыкания
person_1();
person_2();
```

```
</script>
```

Практика применения замыканий

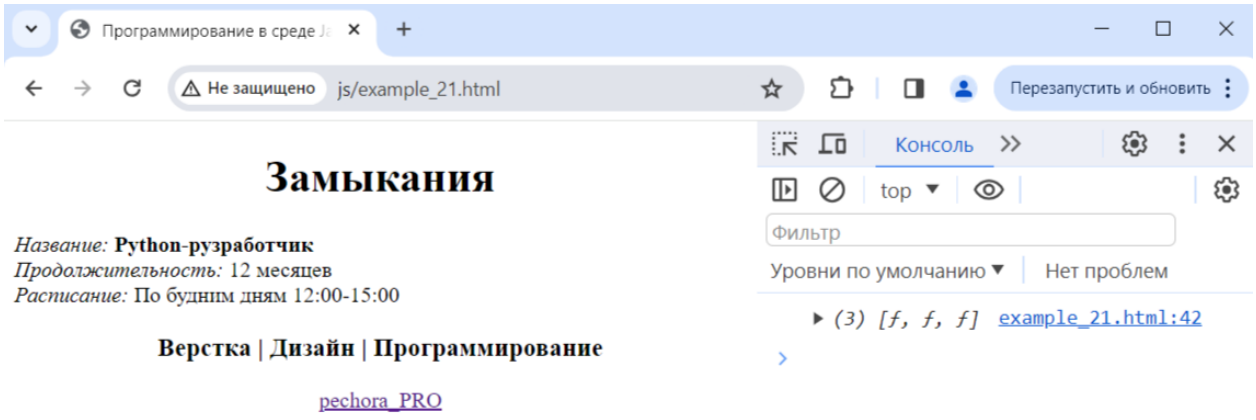
Теперь рассмотрим более приближенные к реальной разработке примеры использования замыканий. В демонстрационном примере **example_21** сохраним в **массив замыкания** интересующих нас курсов.

```
// идентификаторы интересующих нас курсов
let setId = [4, 7, 2];
// создание замыканий
cur = setId.map( (id) => {
    return outCourse(id);
} )
```

example_21.

```
<script>
    // функция поиска курса
    function outCourse(id) {
        let course = courses.find ( (rec) => {
            return rec.id == id;
        } )
        // возвращаем функцию вывода курса
        return function () {
            with (course) {
                document.write(`
                    <i>Название:</i> <b>${name} </b><br>
                    <i>Продолжительность:</i> ${time} <br>
```

```
                <i>Расписание:</i> ${shedule}
            `);
        }
    }
}
// идентификаторы интересующих нас курсов
let setId = [4, 7, 2];
// создание замыканий
cur = setId.map( (id) => {
    return outCourse(id);
} )
// массив функций
console.log(cur);
// пример вывода
cur[2]();
</script>
```



Во всех предыдущих случаях мы сохраняли состояния лексического окружения, но не использовали эти состояния от вызова к вызову.



В демонстрационном примере **example_22** приведу код часто используемого в глобальной сети примера использования счетчика.

example_22.

```
<script>
    // внешняя функция счетчика
    function createCounter() {
        let count = 0;
        // вложенная функция счетчика
        function increment() {
            count += 1;
            return count;
        }
        // возврат текущего значения
        return increment;
    }
    // создаем замыкание
    const counter = createCounter();
    // console.log(counter);
    // вызов замыкания
    console.log(counter()); // 1
    console.log(counter()); // 2
    console.log(counter()); // 3
</script>
```

В **самостоятельных работах** продолжим практику применения замыканий.



Контроль побочных эффектов

Побочный эффект — это изменение состояния программы вне функции. Например, когда функция изменяет значение переменной вне ее области видимости, это будет побочным эффектом. Контроль побочных эффектов подразумевает, что функции **не должны изменять состояние программы вне своей области видимости**.

> ***Примечание***. Функции должны возвращать значения, которые могут понадобиться в других частях программы.

В примере выше **createCounter**() возвращает вложенную функцию **increment**(), которая изменяет **count** внутри своей области видимости. После того, как **createCounter**() была вызвана, создается замыкание, которое сохраняет **count** в памяти и возвращает ссылку на **increment**(). Таким образом, **только во время вызова counter**(), изменяется значение **count** и возвращается новое значение.

Приватные переменные

Замыкания также используются в целях создания приватных переменных и методов.

> ***Примечание***. **Приватные переменные** — это переменные, которые доступны только внутри функции.

Это может быть полезно, когда нужно скрыть какую-то информацию или защитить ее от изменений.

В примере из прошлого раздела была объявлена переменная **count**, которая является приватной, так как она недоступна извне и не может изменяться напрямую.



Для закрепления пройденного материала в **самостоятельных работах** выполним следующие задания:

- Разбор предлагаемого примера кода, поиск ошибки вызова асинхронной функции setTimeout().
- Создание функции, генерирующей и запоминающей максимальное случайное число в диапазоне 1-100.
- Изучение предложенного кода, использующего технологию создания замыкания.
- Написания сценария, создающего и использующего замыкание в поисках объекта массива.

P.S.

Для отработки и закрепления учебного курса **донам** группы предоставляется следующий раздаточный материал.

К каждому уроку курса:

- Файлы **демонстрационного кода** (example);
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой теме курса:

- **Практические** работы.